

# CAP 5. CAMADA DE TRANSPORTE

## AULA 6: PROTOCOLO TCP(CONTINUAÇÃO)

---

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

[ROBERTO.WILLRICH@UFSC.BR](mailto:ROBERTO.WILLRICH@UFSC.BR)

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

# TCP: TEMPO DE RESPOSTA (RTT) E TEMPORIZAÇÃO

---

## Como escolher valor do temporizador TCP?

- maior que o RTT
  - note: RTT pode variar
- muito curto: temporização prematura
  - gera retransmissões desnecessárias
- muito longo: reação demorada à perda de segmentos

# TCP: TEMPO DE RESPOSTA (RTT) E TEMPORIZAÇÃO

---

## Como estimar o RTT

- *SampleRTT*: tempo medido entre a transmissão do segmento e o recebimento do ACK correspondente
- Cada segmento TCP terá seu próprio *sampleRTT*
  - Ignora retransmissões e segmentos reconhecidos de forma cumulativa

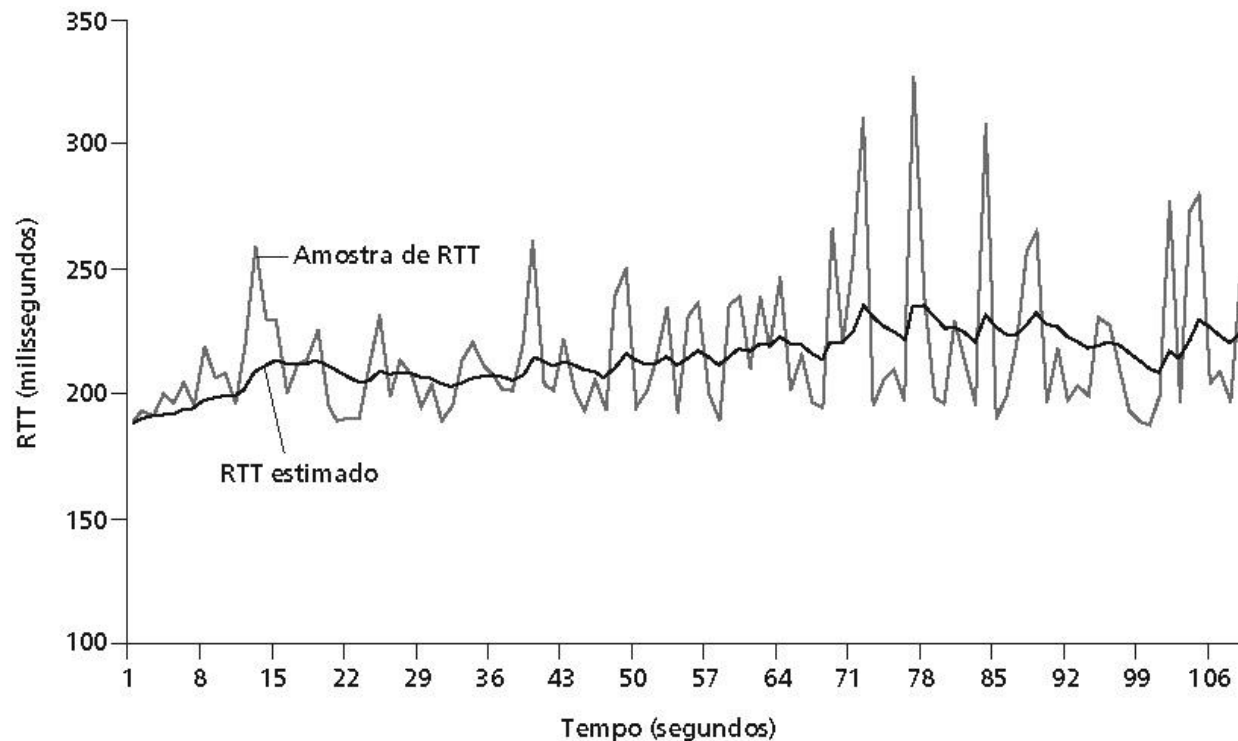
## SampleRTT vai variar devido a congestionamento e sistemas finais

- TCP mantém média de *SampleRTT* (*EstimatedRTT*)

# TCP: TEMPO DE RESPOSTA (RTT) E TEMPORIZAÇÃO

$$\text{EstimatedRTT} = (1-\alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- média corrente exponencialmente ponderada (MMEP)
- valor típico de  $\alpha = 0,125$  (isto é  $1/8$ )

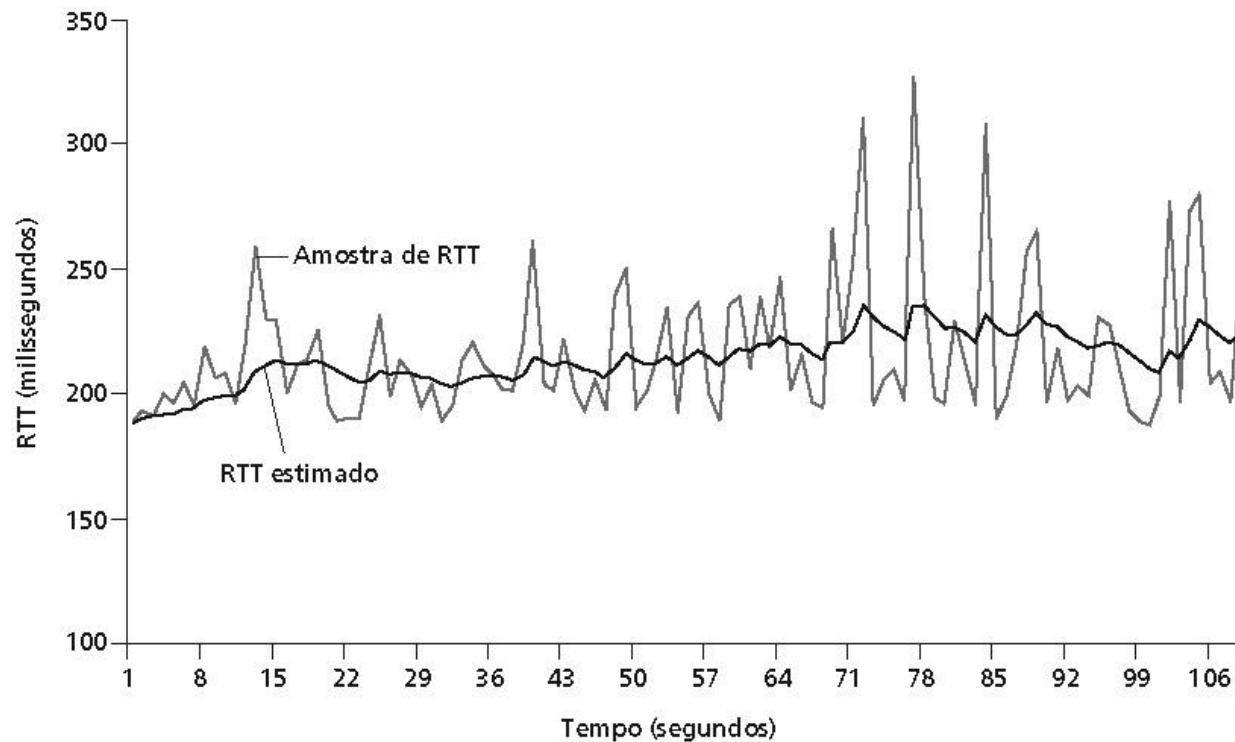


# TCP: TEMPO DE RESPOSTA (RTT) E TEMPORIZAÇÃO

$$\text{EstimatedRTT} = (1 - 0,125) * \text{EstimatedRTT} + 0,125 * \text{SampleRTT}$$

□ RTT0 = 10ms  $\Rightarrow$  EstimatedRTT=10ms

□ RTT1 = 12ms  $\Rightarrow$  EstimatedRTT=  $0,975 * 10 + 0,125 * 12$  ms



# TCP: TEMPO DE RESPOSTA (RTT) E TEMPORIZAÇÃO

---

## Escolhendo o intervalo de temporização

- Intervalo de temporização = EstimatedRTT mais uma “margem de segurança”
- Se há uma variação grande em EstimatedRTT  
-> margem de segurança maior
- Desvio é uma estimativa de quando SampleRTT tipicamente se desvia de EstimatedRTT (MMPE)

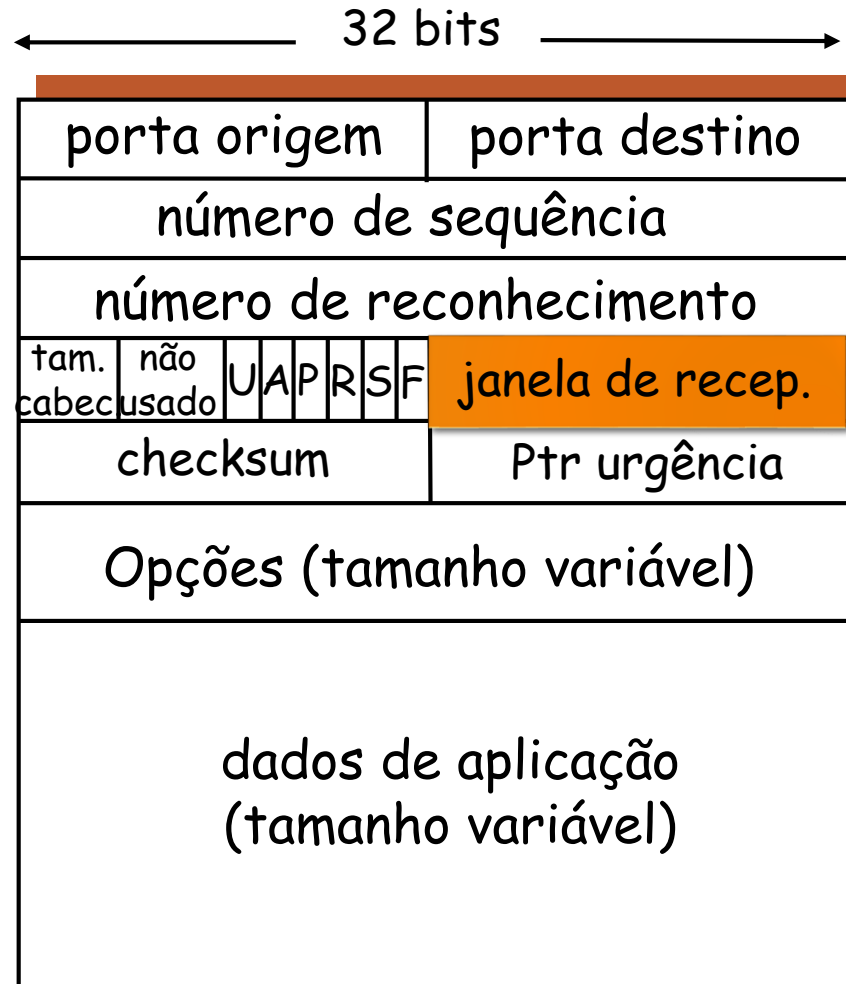
$$\text{Temporização} = \text{RTT\_estimado} + 4 * \text{Desvio}$$

$$\text{Desvio} = (1 - \beta) * \text{Desvio} + \beta * |\text{RTT\_amostra} - \text{RTT\_estimado}|$$

$$\beta = 0.25 \text{ (valor típico)}$$

# ESTRUTURA DO SEGMENTO TCP

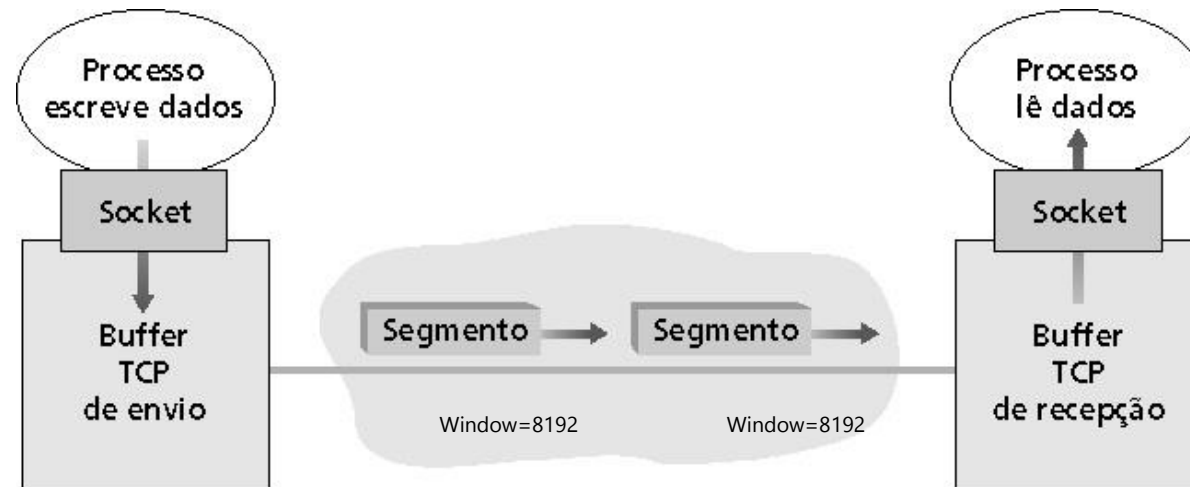
## Controle de fluxo



# PROTOCOLO TCP

## Controle de Fluxo

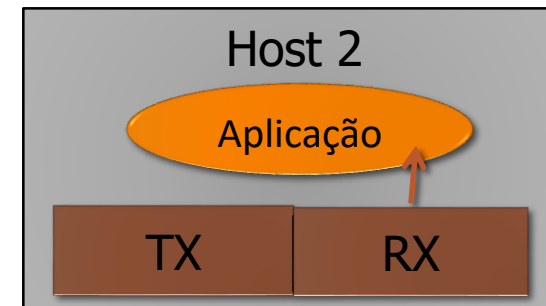
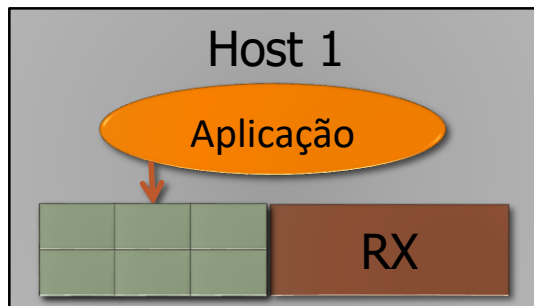
- TCP provê mecanismo para que o transmissor possa determinar o volume de dados que o receptor pode receber
- Baseia-se no campo Window do pacote: o transmissor do pacote informa o número de bytes que o transmissor tem condições de receber
  - O tamanho da janela de recepção (RcvWindow)



# PROTOCOLO TCP

## Controle de Fluxo

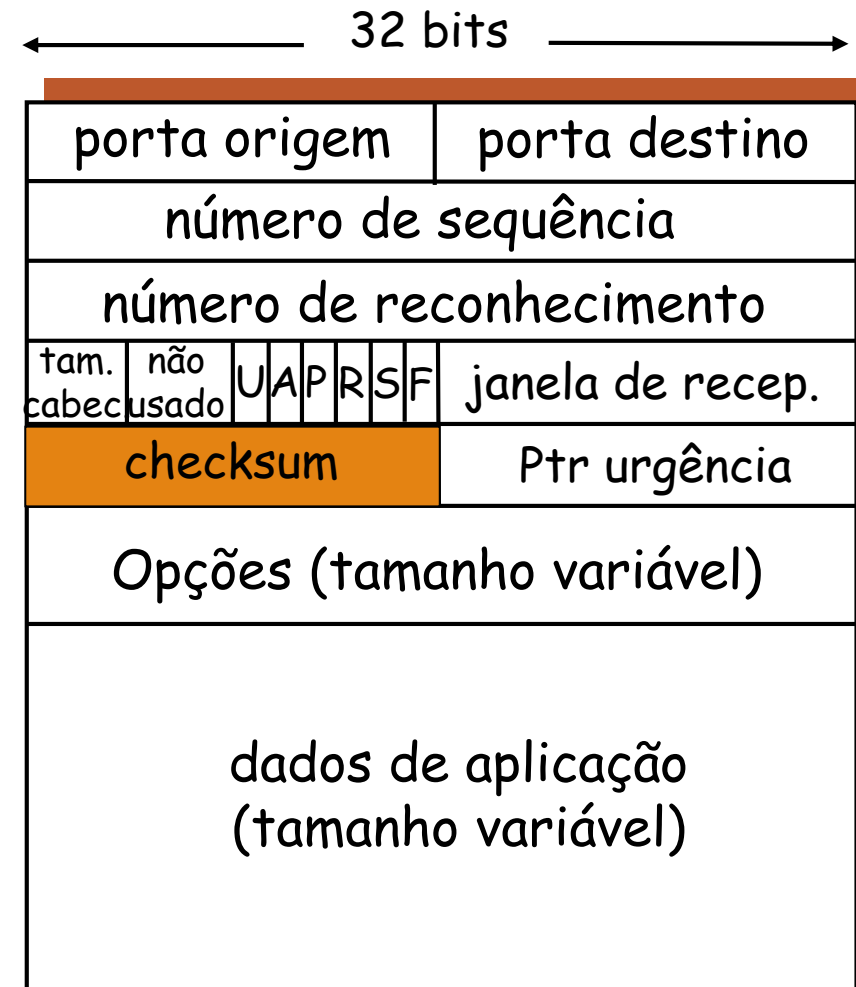
- TCP provê mecanismo para que o receptor possa determinar o volume de dados que o transmissor pode lhe enviar
- Baseia-se no envio, junto com o reconhecimento, do número de octetos que o receptor tem condições de receber contados a partir do último octeto da cadeia de dados recebido com sucesso
- O tamanho da janela de recepção (RcvWindow)



# ESTRUTURA DO SEGMENTO TCP

## Tratamento de erros

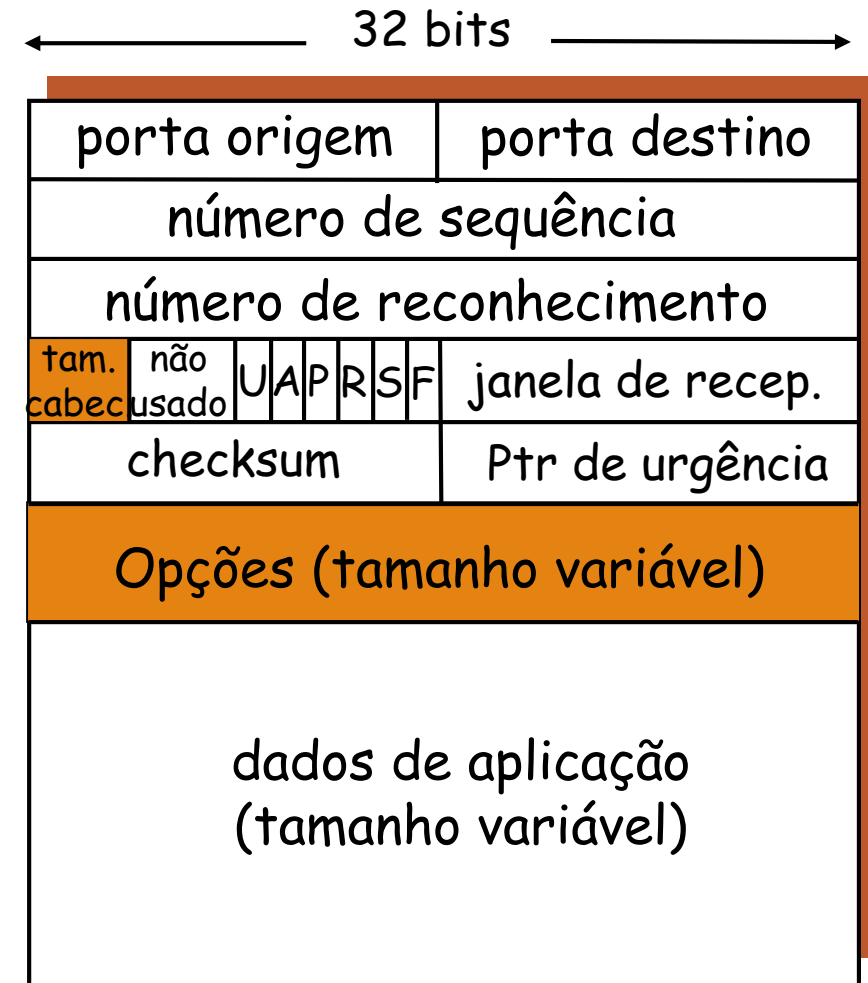
- é adicionado um checksum a cada segmento transmitidos
- Receptor faz uma verificação e os segmentos danificados são descartados



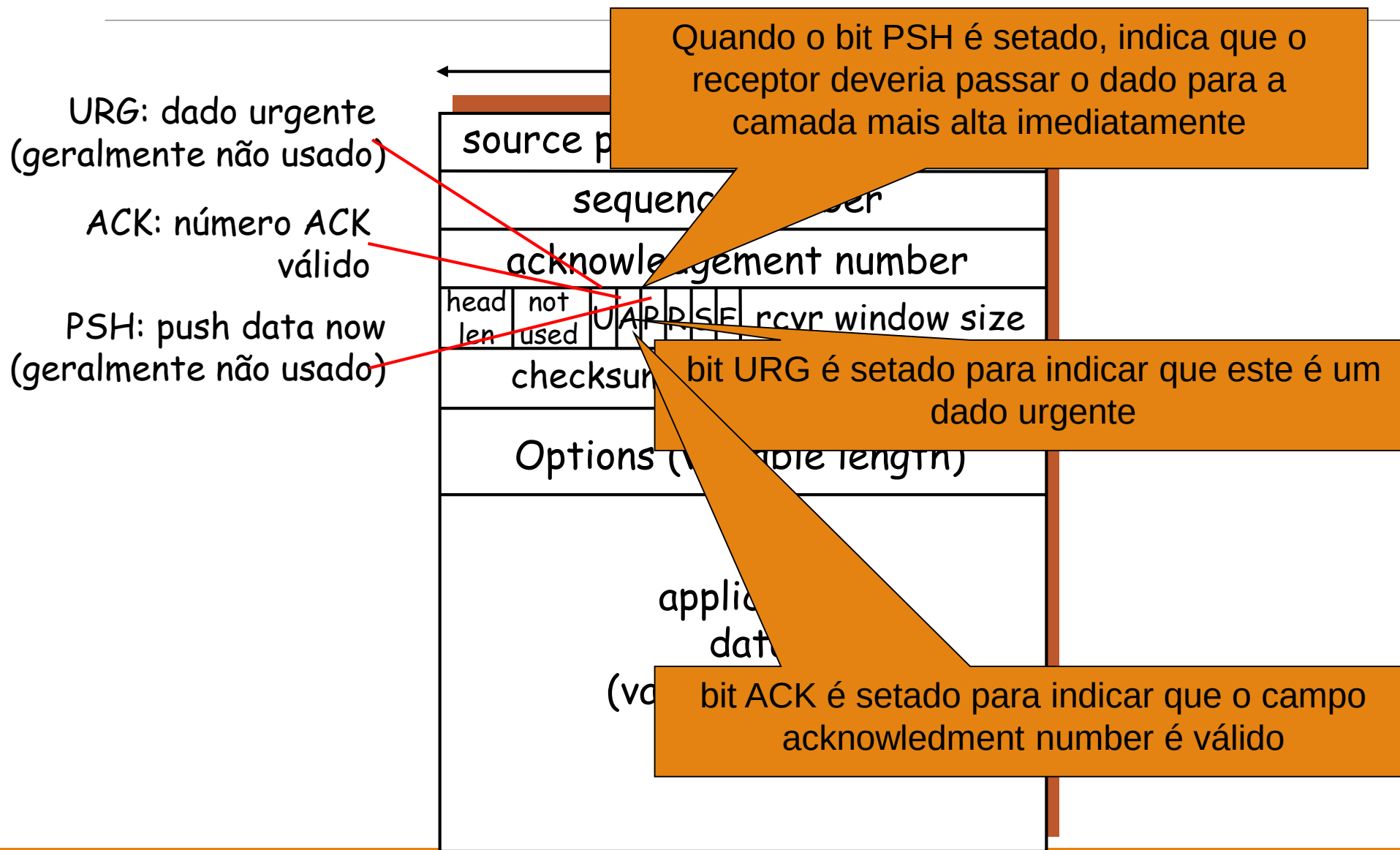
# ESTRUTURA DO SEGMENTO TCP

## Demais Campos

- Campo length (4 bits)
  - Especifica o tamanho do cabeçalho TCP em palavras de 32 bits
  - Cabeçalho tem tamanho variável devido ao campo TCP options
- Campo options
  - Usado quando o emissor e receptor negociam
    - um tamanho máximo de segmento (MSS)
    - fator de escalamento de janela para uso em redes de alta velocidade, para interpretação do campo Window, inicialmente teria um tamanho máximo de 65535 (por ser de 16 bits), sendo que o fator pode ser um multiplicador que permita janelas de até 1GB
  - Uma opção de timestamping também é definida
- Ponteiro de urgência
  - Aponta para o último byte de dados urgente do pacote
- Campo padding
  - Usado quando necessário para completar tamanho de segmento múltiplo de 32 bits



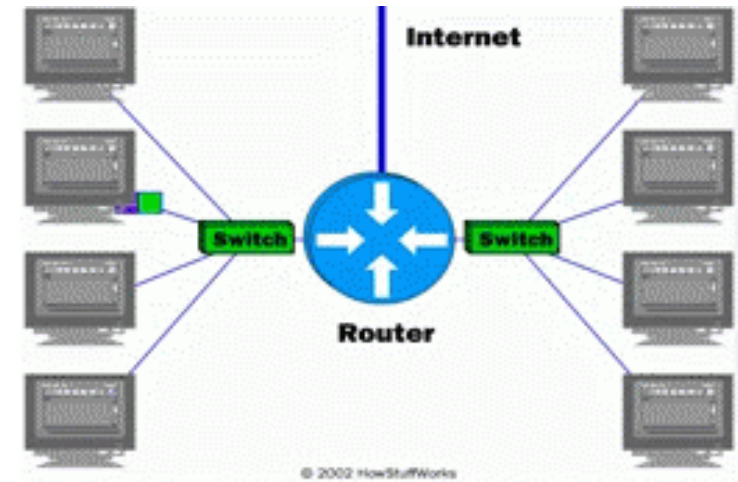
# ESTRUTURA DO SEGMENTO TCP



# CONTROLE DE CONGESTIONAMENTO

## Congestionamento:

- Informalmente: “Excessivo número de fontes enviando grande quantidade de dados mais rápido que a rede possa manipular”
- Manifestações:
  - Pacotes perdidos (overflow dos buffers nos roteadores)
  - Grandes atrasos (enfileiramento nos buffers dos roteadores)
- Um grande problema de rede!



# TCP: CONTROLE DE CONGESTIONAMENTO

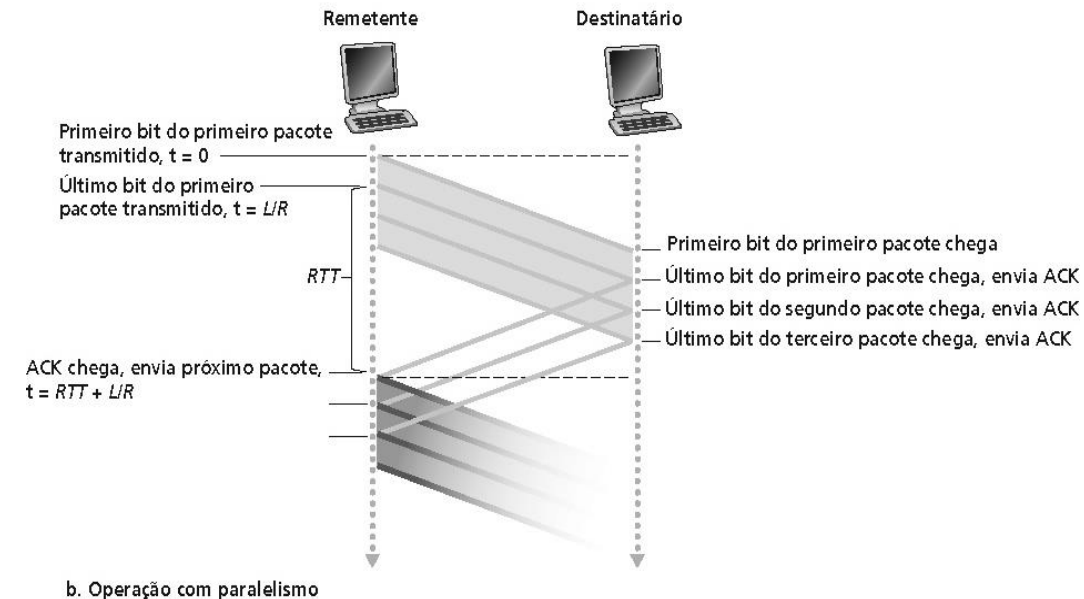
## Controle fim-a-fim (sem assistência da rede)

- Transmissor limita a transmissão:
  - $\text{LastByteSent} - \text{LastByteAcked} \leq \text{CongWin}$

- Aproximadamente,

$$\text{Taxa} = \frac{\text{CongWin}}{\text{RTT}} \text{ Bytes/sec}$$

- CongWin é dinâmico, função do nível de congestionamento da rede
- Como o transmissor detecta o congestionamento?
  - Evento de perda = timeout do temporizador ou 3 ACKs duplicados
- Transmissor TCP reduz a taxa (CongWin) após o evento de perda



# CONTROLE DE CONGESTIONAMENTO TCP

---

## Partida Lenta

- Quando a conexão começa, CongWin = 1 MSS
  - Exemplo: MSS = 500 bytes e RTT = 200 milissegundos
    - Taxa inicial = 20 kbps
- Largura de banda disponível pode ser  $\gg$  MSS/RTT
  - Desejável aumentar rapidamente até a taxa respeitável
  - Quando a conexão começa, a taxa aumenta rapidamente de modo exponencial até a ocorrência do primeiro evento de perda
- Na partida lenta a banda oferecida cresce exponencialmente até chegar a um limiar ou ocorrência de perda
  - Na ocorrência da perda recomeça a partida lenta

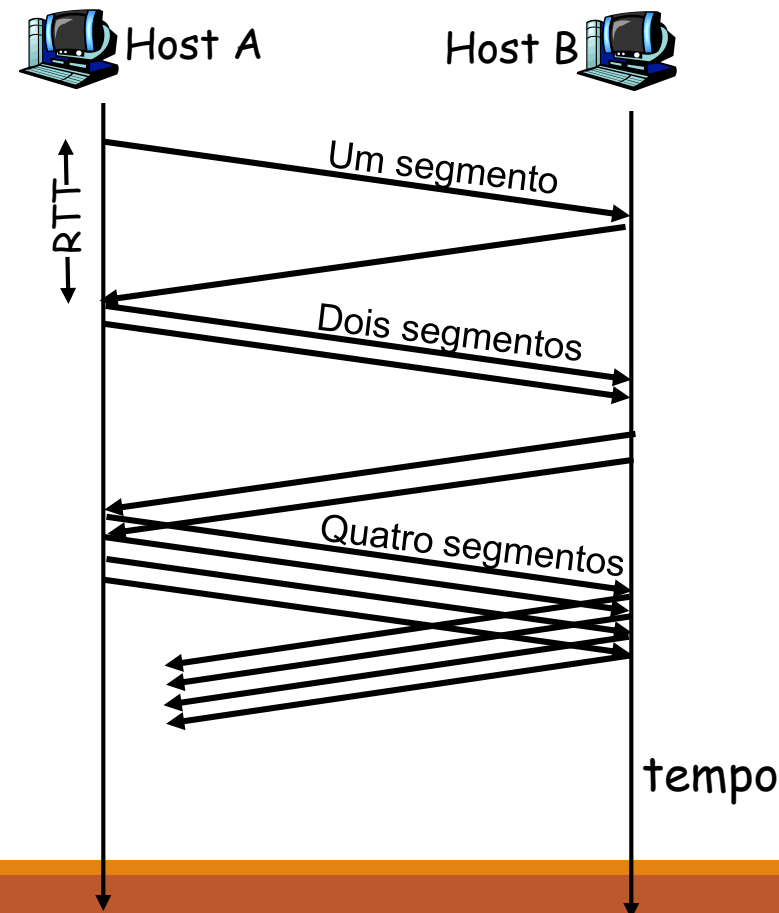
# PARTIDA LENTA TCP

**Incremento exponencial no tamanho da janela (não muito lenta!)**

**Evento de perda: timeout e/ou três ACKs duplicados**

## Algoritmo Partida lenta

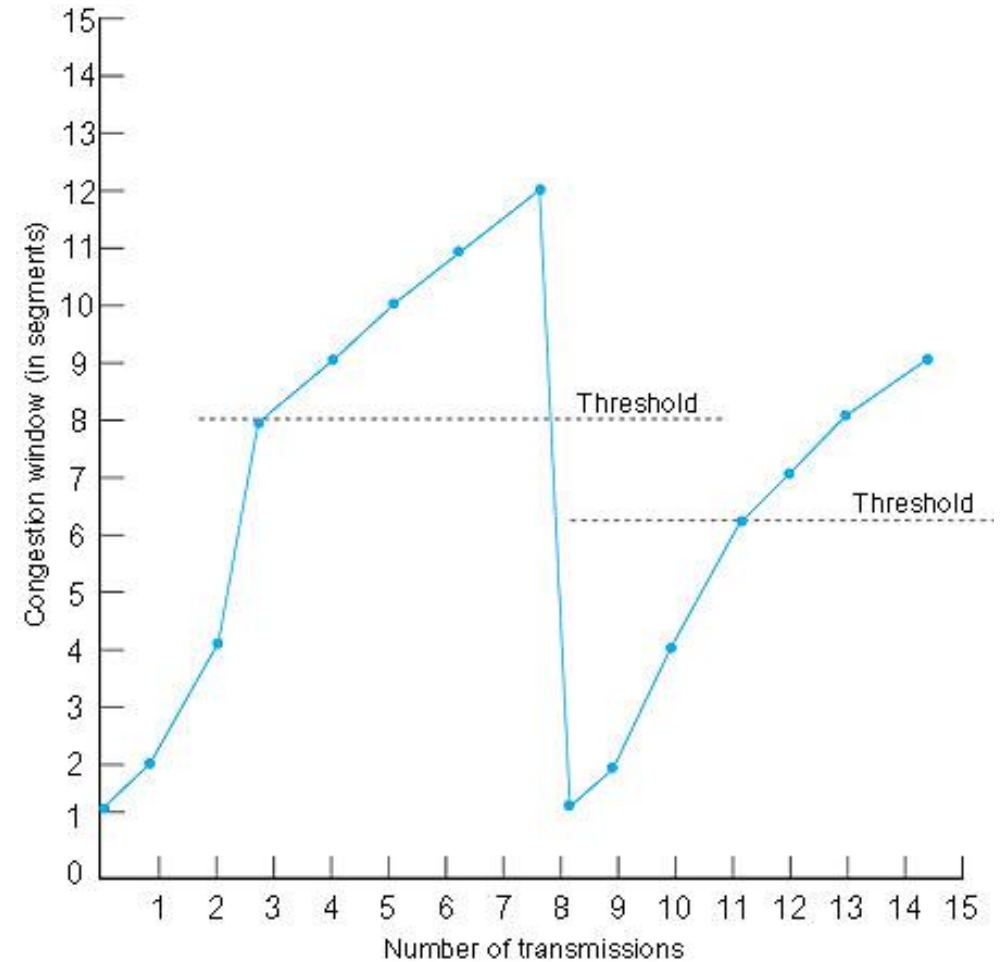
inicializa: Congwin = 1 MSS  
Para (cada segm com ack)  
    Congwin++  
Até (evento de perda OU  
    CongWin > threshold)



# CONTROLE DE CONGESTIONAMENTO TCP

## Fase de partida lenta

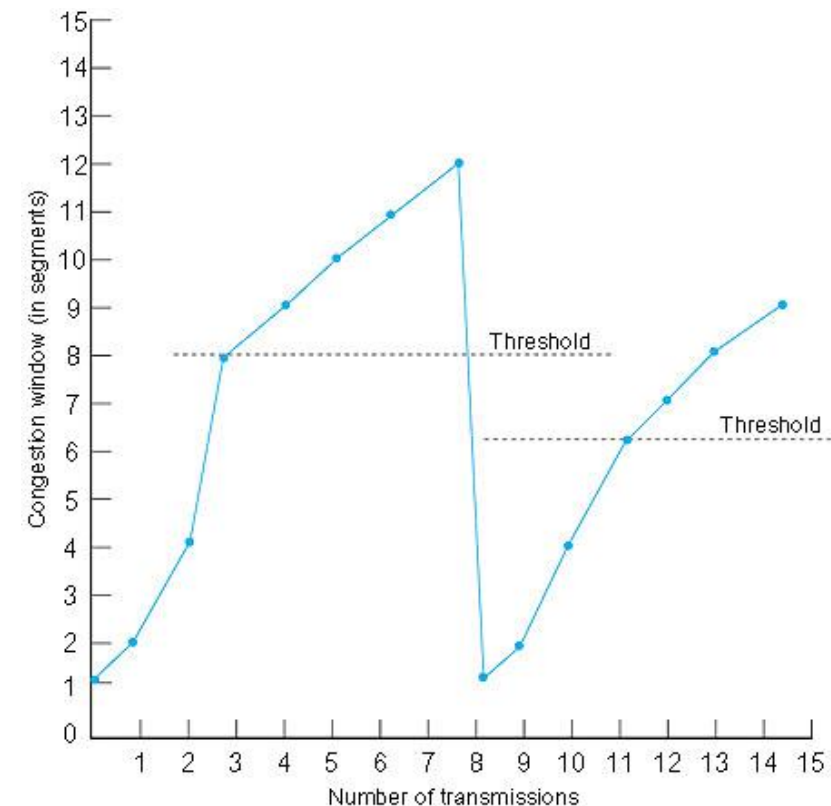
- Termina quando o tamanho da janela excede o valor do threshold
- Uma vez que a janela de congestionamento é maior que o valor atual do threshold, a janela de congestionamento cresce linearmente (e não mais exponencialmente)
- Esta fase é chamada de prevenção do congestionamento



# CONTROLE DE CONGESTIONAMENTO TCP

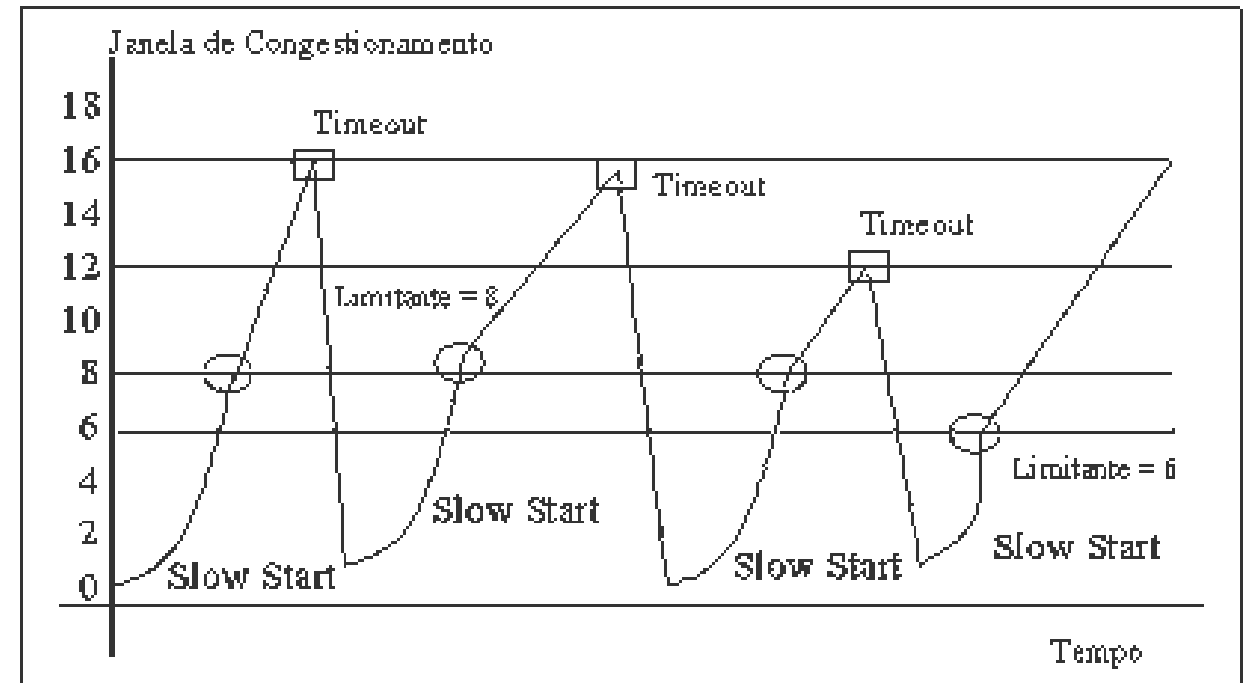
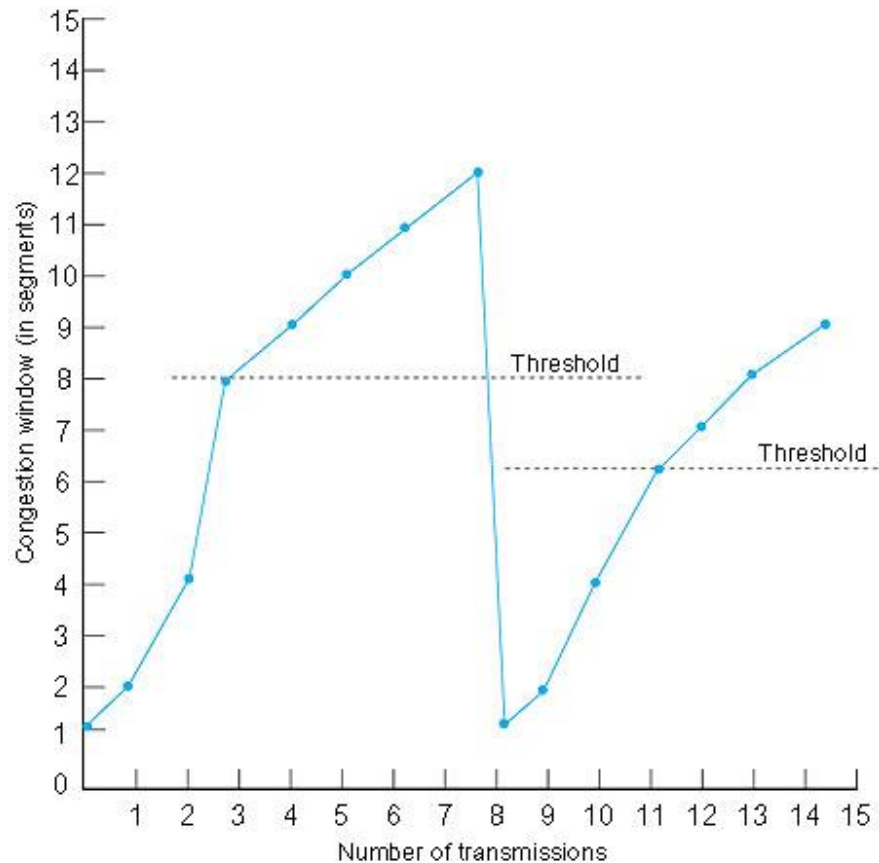
## Fase de prevenção de congestionamento

- Mas o tamanho da janela não podem crescer para sempre
- Eventualmente, a taxa TCP será tal que um dos enlaces seja saturado
  - Em que perdas ocorrerão (e resultante timeout no emissor)
- Quando um timeout ocorrer
  - threshold é setado como a metade do valor da janela de congestionamento atual, e a janela de congestionamento é resetada para um MSS
  - Emissor então procede o incremento exponencial da janela de congestionamento usando o procedimento de partida lenta até a janela de congestionamento alcançar o threshold



# CONTROLE DE CONGESTIONAMENTO TCP

## Fase de prevenção de congestionamento



# CONTROLE DE CONGESTIONAMENTO TCP

---

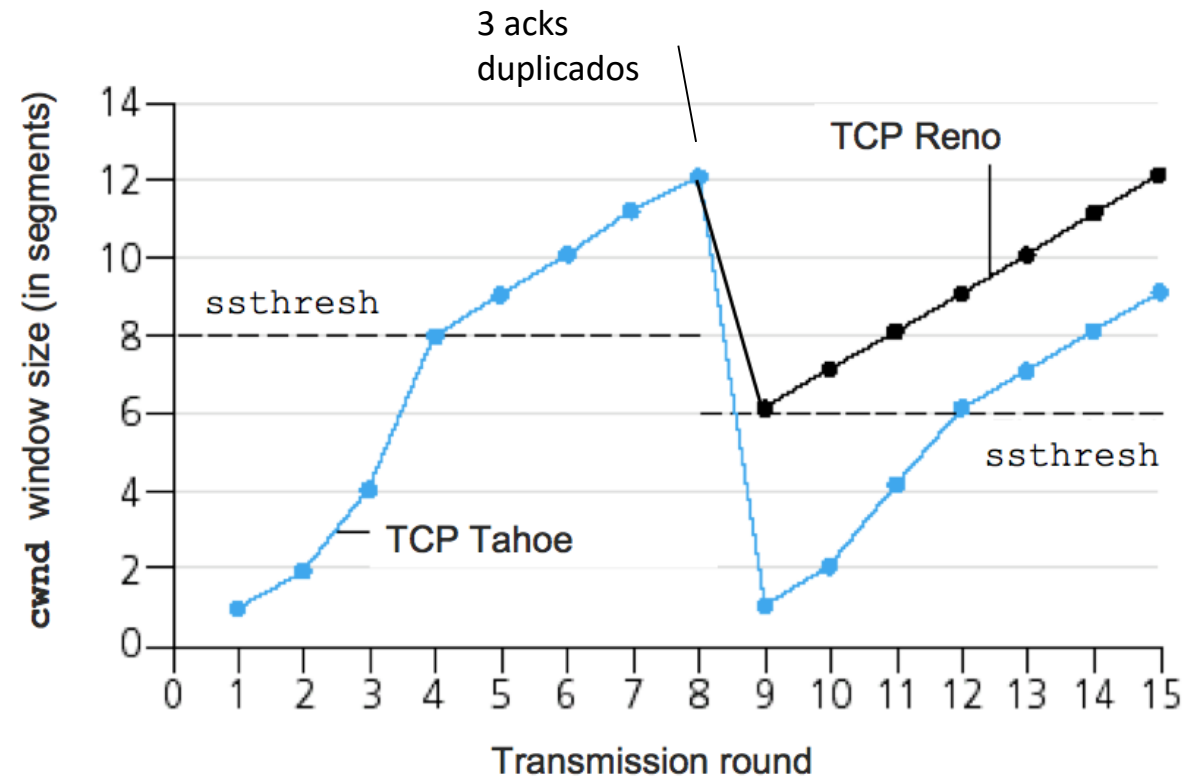
## Algoritmo Tahoe

- Primeira implementação, com algoritmos de Partida Lenta, Prevenção de Congestionamento e retransmissão rápida
  - Retransmissão rápida: 3 acks duplicados ocasionam retransmissão do pacote e retorno a partida lenta.
- Com muitas perdas provoca muitos retornos à partida lenta

## Algoritmo Reno

- Opera melhor em redes com muitas perdas
- Tem mecanismo de recuperação rápida que cancela a fase de partida lenta após uma retransmissão rápida

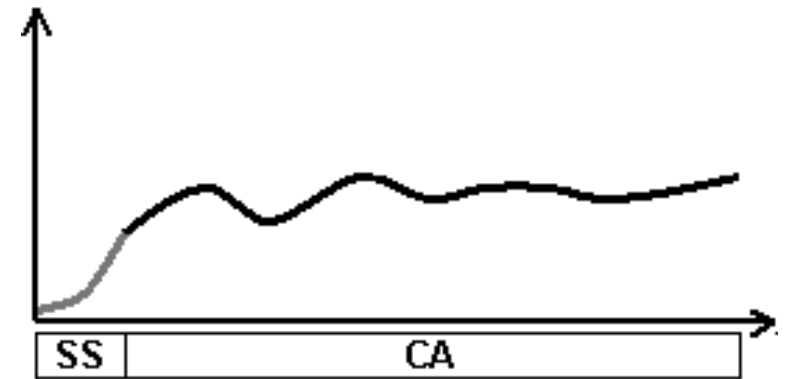
# CONTROLE DE CONGESTIONAMENTO TCP



# CONTROLE DE CONGESTIONAMENTO TCP

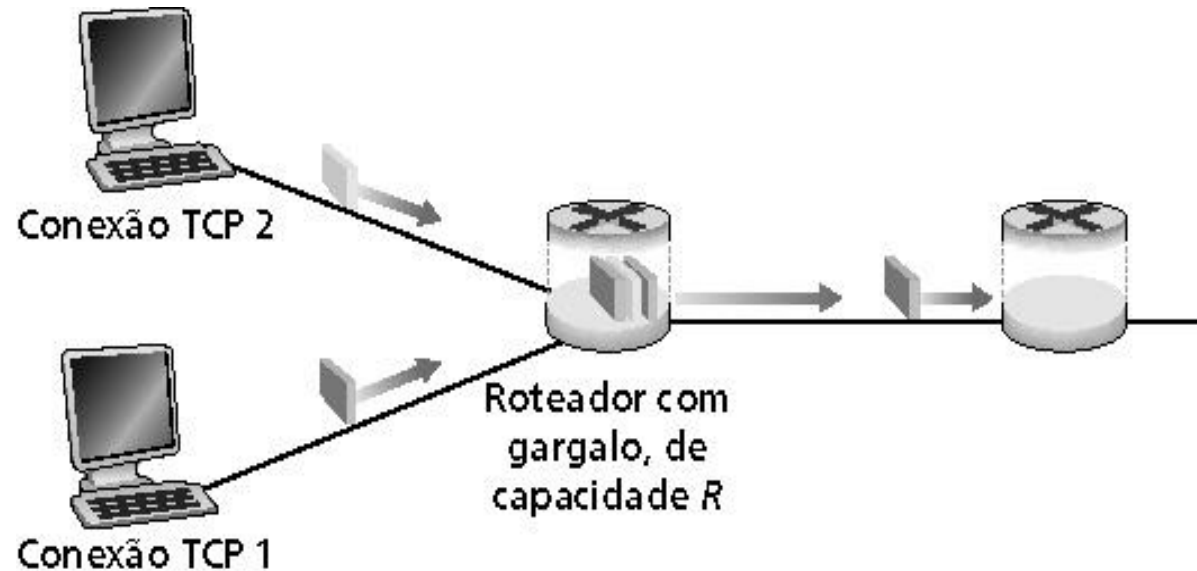
## Algoritmo Vegas

- Aumenta o desempenho do Reno
- Tenta evitar o congestionamento mantendo uma boa vazão
- Ideia básica
  - detectar o congestionamento nos roteadores entre a fonte e destino antes da ocorrência da perda
  - reduz a taxa linearmente quando a perda eminente de pacotes é detectada
  - Perda eminente é prevista pela observação do atraso de ida-e-volta
    - Maior o atraso de ida-e-volta dos pacotes, maior o congestionamento nos roteadores



# EQUIDADE DO TCP

**Objetivo de equidade: se  $K$  conexões TCP compartilham o mesmo enlace de gargalo com largura de banda  $R$ , cada uma deve ter taxa média de  $R/K$**



# EQUIDADE DO TCP

---

## Equidade e conexões TCP paralelas

- Conexões no computador tendem a receber a mesma vazão
- Nada previne as aplicações de abrirem conexões paralelas entre 2 hospedeiros
  - Web browsers fazem isso
- Exemplo: enlace de taxa R suportando 9 conexões;
  - Nova aplicação pede 1 TCP, obtém taxa de  $R/10$
  - Nova aplicação pede 11 TCPs, cada uma obtém  $R/20$ !

## Equidade e UDP

- Aplicações multimídia normalmente não usam TCP
  - Não querem a taxa estrangulada pelo controle de congestionamento
- Em vez disso, usam UDP:
  - Trafega áudio/vídeo a taxas constantes, toleram perda de pacotes